Carson Hansen
10/19/25

HW 1 K-NN for Income Prediction

1. A
    1.1. There were 1251 data points in the training data that were > 50K. Meaning that about 25% of the data is over 50k. The dev set had 236 data points that were >50K. Meaning that the set had about 23.6% of the data had people making >50k. I would say that this data does not make sense given my knowledge of the average US per capita income. I was under the impression that the average income was about 50k per year. Having only about 25% of the data from both sets is much lower then what I was expecting to see.
    1.2. The youngest is 17 years old and the oldest is 90.The least amount of hours worked is 1, while the most is 90.
    1.3.
        1.3.1. You need to binarize all categorical fields because there is no significant numerical value in categorical fields. By binarizing these fields you avoid false ordering of categories by keeping them 0 and 1.
        1.3.2. You don't want to binarize fields such as age and hours because their numeric values are meaningful. Binarizing these fields would make values such as 20 and 35 indistinguishable and not meaningful.
    1.4. For two numerical fields, I feel that the best approach would be to normalize them. This would make these numerical fields be more in line with the other categories and would reduce the chances of these values having too much influence on the model if they got too big. Since the max age is 90, the max distance on age would be .9 if it was normalized from 0,1 0r 1.8 if it was 0,2. The fields are not treated equally if they are scaled, since some values can be something besides 0 or 1.
    1.5. There are a total of 90 features. 7 sectors, 16 education, 7 types of marital status, 14 occupations, 5 races, 2 sexes, and 39 countries.
2.

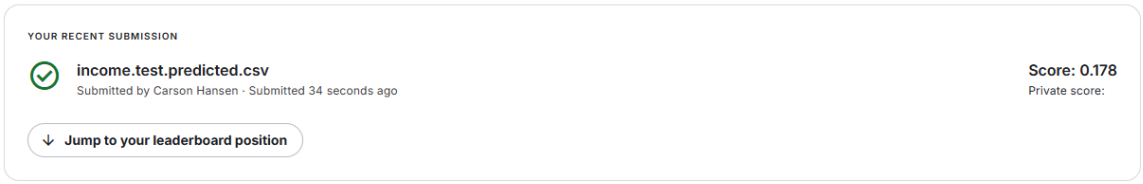
2.2. You can't use it for machine learning because not all fields may line up. This will cause all data to be off and will not produce the desired outcome or learning objective. You may end up with different feature dimensions that will throw everything off.

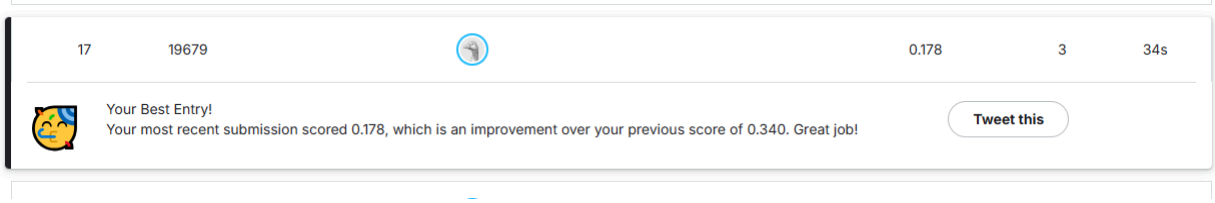
2.3. The feature dimension is 92. This does not match with what I got for 1.5. I got 90 for that one.

2.4.a. The best dev error I get is 19.20% k = 49.

2.4.b. When k=1, training error should be 0%. This is because the closest neighbor should be itself. This does not line up with my data. I get a 1.2% train error.

2.4.c I notice that as k increases, the error rates get better when you increase k. Once k is somewhat big, the error rate doesn't change too much. The running speed increases when k increases.
2.4.d.When k = inf, that means that any point or new point that gets added will always be a nearest neighbor. So you are accounting for all data points in the set. This would be an example of extreme underfitting since the model would be very biased and insensitive to local data patterns and won't pick up on data nuances. For k =1, this means that the closest neighbor would be itself. This would be an example of extreme overfitting since there is not enough data to go off of and any noise or outliers can mislabel points.
2.4.e My error rate was 17.8%, which at this time, put me in 17th place.



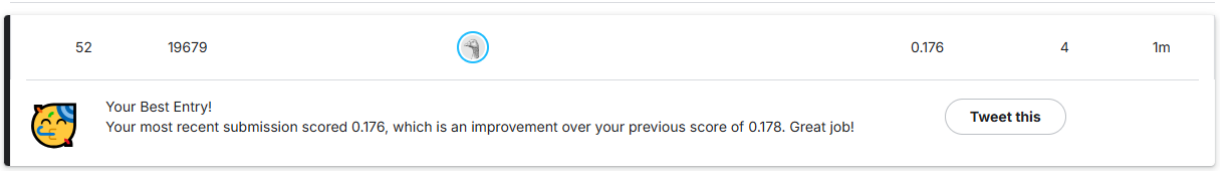


3.

3.1. My best error rate on dev was 24% at k = 49. I did not notice any performance improvements compared to my initial results. I think this may be due to the fact that this model does not have proper scaling in the values for age and hours. I feel that this throws some things off since having imbalance weights can greatly influence certain fields.

3.2 My best error rate on dev was 19.70% at k = 33. I noticed a significant performance increase when compared to the initial results. I believe this is because the scaling is a lot better than before. Fields that were dominating didn't have as much pull in this approach compared to the first iteration.

3.3 My error rate was 17.6% and at the time of this, I was ranked 52nd.



4.

4.1 For Sklearn, The distances I got were .334, 1.415, and 1.417 for Euclidean.These values correspond to the people with ID 4872,4787, and 2591. For Manhattan, I got .399, 2.055, and 2.102. These values correspond to the people with ID 4872, 4787, and 1084.

4.2.a. No, there is no more work in the training after the feature map has been completed. Once the feature map is complete, there are no updated parameters that need to be learned upon.

4.2.b. The time complexity of k-nn is O(d * |D|). This is because the worst case would be taking the number of (d) features and would cause you to have to compute the distance of every sample D there is. So in the end, it would be O(d * |D|).

4.2.c. No, you don't really need to sort the distance first to choose the top k value. You can do a partial sort to get this data. For this I used the ***np.argpartition()*** to help me out.

4.2.d. The numpy tricks I used to speed things up were broadcasting, np.argsort and np.argpartition.

4.2.e It took 4.362 user seconds to print the training and dev errors for k =99.

4.3. It took 1 minute and 18.894 user seconds to print k = 1 ~ 99 when using the manhattan approach. This was significantly worse in time when compared to the euclidean version. I did expect this since it's computing ~50 k values instead of 1. I also noticed that it took significantly longer to run each iteration compared to the euclidean version. When comparing the error rates of k =99, the euclidean version had a 19.90% dev error while the manhattan had 19.80%. So, in terms of which model was better based on dev error rate at k = 99, manhattan seems to be slightly better then the euclidean version.

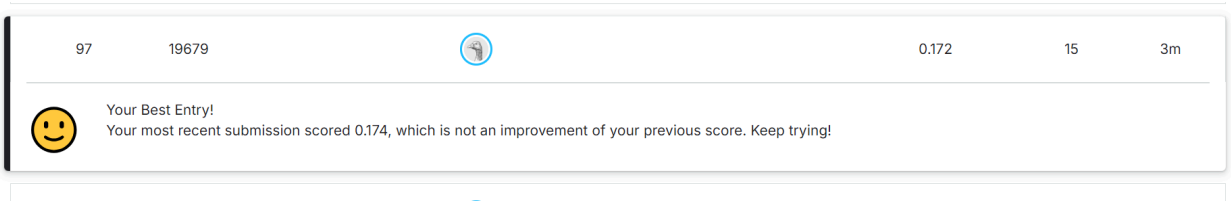
5.

5.1. My best K was 55 using the euclidean approach.

5.2 My best dev error rate was 18.40% and the corresponding positive ratio is 18.7%

5.3 The positive ratio on test was also 18.7%

5.4 My best rank was 97th with 15 submissions



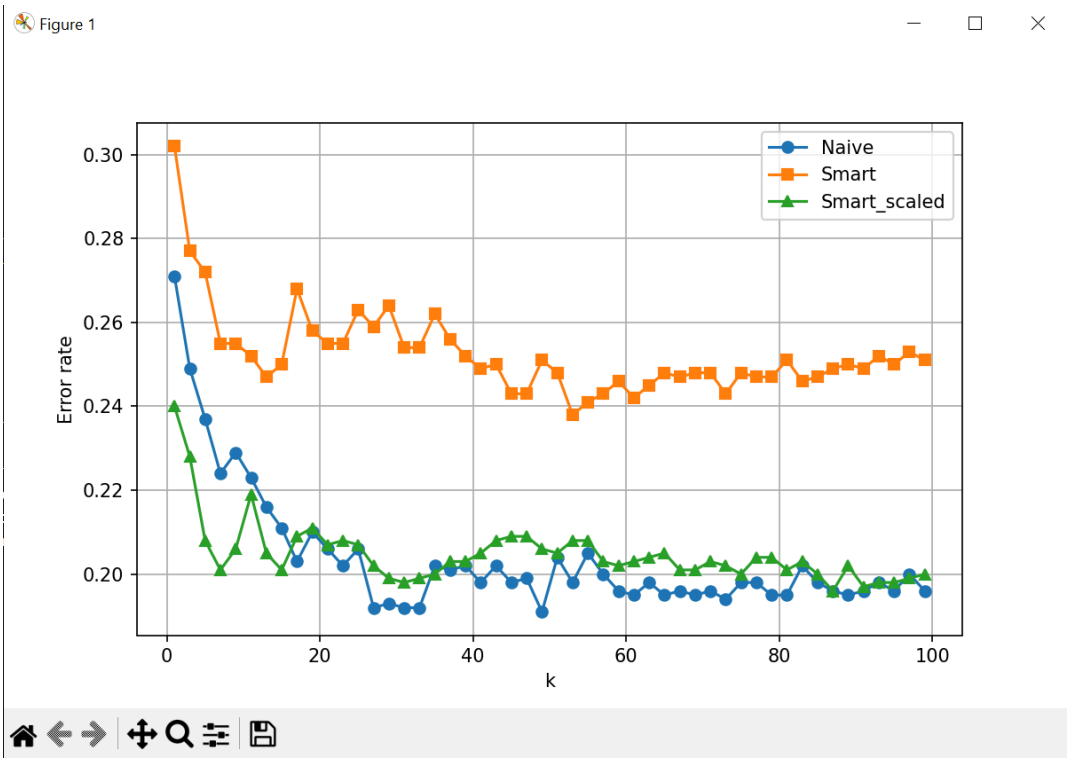
6

6.1 k-NN treats all features equally, even though some (like education or hours worked) would be some of the most important data pieces when predicting income. It is also sensitive to feature scaling so you need to make sure all data is properly scaled to avoid any feature from scaling the data. k-NN can also be very slow on large dimensional datasets since it will need to take everything into consideration.

6.2 Yes, I would say that in this HW that the best-performing models did tend to exaggerate existing bias in the training data. Out of the 1000 people in the dataset, only 3.77% of men and 28.59% of women made above 50k. I would say that this is caused by an underfitting and not an overfitting. I don't think I would go as far to say that it is because of a

social issue and could be because of the training data. When analyzing the training data, I noticed that out of the 5,000 samples there was a 30.66% chance for men and 12.94% chance for women to make above 50k. My guess to why such little women are predicted to be above 50k would be because of the training data. There was not much for the model to train off of to really determine whether or not

7.



Debrief:

1. I would say I spent about 25 hours on this assignment.
2. I would rate this as a moderate assignment. I have no background in machine learning and this was my first exposure to it. So I needed to do a bit of background digging to grasp some concepts. The assignment wasn't too challenging until it came to implementing our own k-nn algorithm. That was probably the most challenging portion, but it wasn't too bad to do.
3. I worked on this assignment on my own.
4. I would say that I'm at about an 80-85% of material understanding
5. This was a pretty cool topic to introduce machine learning. I enjoyed learning about k-nn and how it works. I did like the code snippets that were provided. It

would be nice if some pseudo-code was provided for when things got a bit more involved. I just want to make sure I’m on the right track. The hints that were provided did help a ton!